

Python Functions

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.

Creating a Function

- In Python a function is defined using the `def` keyword:

Example

```
def my_function():  
    print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

Example

```
def my_function():  
    print("Hello from a function")  
  
my_function()
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

Example

```
def my_function(fname):  
    print(fname + " Refsnes")  
  
my_function("Emil")  
my_function("Tobias")  
my_function("Linus")
```

1) Default Arguments

A default contention is a boundary that takes as information a default esteem, assuming that no worth is provided for the contention when the capability is called. The following example demonstrates default arguments.

```
# Python code to demonstrate the use of default arguments  
# defining a function  
def function(n1, n2=20):  
    print("number 1 is: ", n1)  
    print("number 2 is: ", n2)  
# Calling the function and passing only one argument  
print("Passing only one argument")  
function(30)  
  
# Now giving two arguments to the function  
print("Passing two arguments")  
function(n1: 50, n2: 30)
```



```
Passing only one argument
number 1 is: 30
number 2 is: 20
Passing two arguments
number 1 is: 50
number 2 is: 30
```

2) Keyword Arguments

Keyword arguments are linked to the arguments of a called function. While summoning a capability with watchword contentions, the client might tell whose boundary esteem it is by looking at the boundary name. We can eliminate or orchestrate specific contentions in an alternate request since the Python translator will interface the furnished watchwords to connect the qualities with its boundaries. One more method for utilizing watchwords to summon the capability() strategy is as per the following:

```
# Python code to demonstrate the use of keyword arguments
# Defining a function
def function(n1, n2):
    print("number 1 is: ", n1)
    print("number 2 is: ", n2)

# Calling function and passing arguments without using keyword
print("Without using keyword")
function(n1: 50, n2: 30)

# Calling function and passing arguments using keyword
print("With using keyword")
function(n2=100, n1=700)
```

```
Without using keyword
number 1 is: 50
number 2 is: 30
With using keyword
number 1 is: 700
number 2 is: 100
```

3) Required Arguments

Required arguments are those supplied to a function during its call in a predetermined positional sequence. The number of arguments required in the method call must be the same as those provided in the function's definition.

We should send two contentions to the capability() all put together; it will return a language structure blunder, as seen beneath.

```
# Defining a function
def function(n1, n2):
    print("number 1 is: ", n1)
    print("number 2 is: ", n2)

# Calling function and passing two arguments out of order, we need num1 to be 20 and num2 to be 30
print("Passing out of order arguments")
function( n1: 30, n2: 20)

# Calling function and passing only one argument
print("Passing only one argument")
try:
    function(30)
except:
    print("Function needs two positional arguments")
```

```
C:\Users\praj1\PycharmProjects\pythonProject11
Passing out of order arguments
number 1 is: 30
number 2 is: 20
Passing only one argument
Function needs two positional arguments
```

4) Variable-Length Arguments

We can involve unique characters in Python capabilities to pass many contentions. However, we need a capability. This can be accomplished with one of two types of characters:

"args" and "kwargs" refer to arguments not based on keywords.

Eg1:

```
# Python code to demonstrate the use of variable-length arguments
# Defining a function
def function( *args_list ):
    ans = []
    for l in args_list:
        ans.append( l.upper() )
    return ans
# Passing args arguments
object = function( *args_list: 'Python', 'Functions', 'tutorial' )
print( object )
```

```
C:\Users\praji\PycharmProjects\pythonProject11\
['PYTHON', 'FUNCTIONS', 'TUTORIAL']
```

Eg2:

```
# defining a function
def function( **kwargs_list ):
    ans = []
    for key, value in kwargs_list.items():
        ans.append([key, value])
    return ans
# Paasing kwargs arguments
object = function(First = "Python", Second = "Functions", Third = "Tutorial")
print(object)
```

```
C:\Users\praji\PycharmProjects\pythonProject11\.venv\Scripts\python.exe C:\U
[['First', 'Python'], ['Second', 'Functions'], ['Third', 'Tutorial']]
Process finished with exit code 0
```

return Statement

When a defined function is called, a return statement is written to exit the function and return the calculated value.

Syntax:

```
return < expression to be returned as output >
```

The return statement can be an argument, a statement, or a value, and it is provided as output when a particular job or function is finished. A declared function will return an empty string if no return statement is written.

```
# Python code to demonstrate the use of return statements
# Defining a function with return statement
def square(num):
    return num ** 2

# Calling function and passing arguments.
print("With return statement")
print(square(4))
```



```
C:\Users\praji\PycharmProje
With return statement
16

Process finished with exit
```

The pass Statement

`function` definitions cannot be empty, but if you for some reason have a `function` definition with no content, put in the `pass` statement to avoid getting an error.

Example

```
def myfunction():  
    pass
```

Positional-Only Arguments

You can specify that a function can have ONLY positional arguments, or ONLY keyword arguments.

To specify that a function can have only positional arguments, add `, /` after the arguments:

Example

```
def my_function(x, /):  
    print(x)  
  
my_function(3)
```

Without the `, /` you are actually allowed to use keyword arguments even if the function expects positional arguments:

Example

```
def my_function(x):  
    print(x)  
  
my_function(x = 3)
```

But when adding the `, /` you will get an error if you try to send a keyword argument:

Example

```
def my_function(x, /):  
    print(x)  
  
my_function(x = 3)
```

Scope and Lifetime of Variables

A variable's scope refers to the program's domain wherever it is declared. A capability's contentions and factors are not external to

the characterized capability. They only have a local domain as a result.

The length of time a variable remains in RAM is its lifespan. The lifespan of a function is the same as the lifespan of its internal variables. When we exit the function, they are taken away from us. As a result, the value of a variable in a function does not persist from previous executions.

An easy illustration of a function's scope for a variable can be found [here](#).

```
# Python code to demonstrate scope and lifetime of variables
# defining a function to print a number.
def number():
    num = 50
    print("Value of num inside the function: ", num)

num = 10
number()
print("Value of num outside the function:", num)
```

```
Value of num inside the function: 50
Value of num outside the function: 10
```

Recursion

Any function that calls itself in its body repeatedly until a particular condition becomes false and the target task is done is called a "Recursive function" and this procedure is called "Recursion".

Syntax:

```
def function_name(arguments if any):  
    #function body  
    function_name(argument) #recursive call of the function
```

```
def recurse():  
    ...  
    recurse()  
    ...  
  
recurse()
```

```
#function  
def fact(num):  
    if(num==1 or num==0):  
        return 1  
    else:  
        return(num*fact(num-1))  
  
#Body of the program  
num = int(input("Enter a number: "))  
if(num<0):  
    print("Invalid number")  
else:  
    result = fact(num)  
    print("The factorial of the given number", num, "is", result)
```

```
C:\Users\praji\PycharmProjects\pythonProject11\.venv
Enter a number: 5
The factorial of the given number 5 is 120
Process finished with exit code 0
```

Python Built-in Functions

The Python built-in functions are defined as the functions whose functionality is pre-defined in Python. The python interpreter has several functions that are always present for use. These functions are known as Built-in Functions. There are several built-in functions in Python which are listed below:

- 1.
2. The python **abs()** function is used to return the absolute value of a number.

```
# integer number
integer = -20
print('Absolute value of -20 is:', abs(integer))

# floating number
floating = -20.83
print('Absolute value of -20.83 is:', abs(floating))

#complex
compl=1+2j
print("Absolute value of complex number ",compl," is:",abs(compl))
```

```
C:\Users\praji\PycharmProjects\pythonProject11\.venv\Scripts\python.exe
Absolute value of -20 is: 20
Absolute value of -20.83 is: 20.83
Absolute value of complex number (1+2j) is: 2.23606797749979
```

3. The python **all()** function accepts an iterable object (such as list, dictionary, etc.). It returns true if all items in passed iterable are true. Otherwise, it returns False

```
# all values true
k = [1, 3, 4, 6]
print(k, " is returns:", all(k))

# all values false if contain false
k = [0, False]
print(k, "is returns:", all(k))

#true-if empty
k = []
print(k, "is returns:", all(k))
```

```
[1, 3, 4, 6] is returns: True
[0, False] is returns: False
[] is returns: True
```

4. The python **bin()** function is used to return the binary representation of a specified integer. A result always starts with the prefix 0b.

```
#binary of integer srating with 0b
a=5
print("binary of ",a,"is :",bin(a))
```

```
C:\Users\praji\PycharmProject
binary of 5 is : 0b101
```

5. The python **bool()** converts a value to boolean(True or False) using the standard truth testing procedure.

```
test1 = []  
print(test1,'is',bool(test1))  
test1 = [0]  
print(test1,'is',bool(test1))
```



```
C:\Users\praji\Pyd  
[] is False  
[0] is True
```

6. The python **any()** function returns true if any item in an iterable is true. Otherwise, it returns False.

```
l = [4, 3, 2, 0]  
print(any(l))  
  
l = [0, False]  
print(any(l))
```



```
C:\Users\praj  
True  
False
```

7. The python **float()** function returns a floating-point number from a number or string.

```
# for integers
print(float(9))

# for floats
print(float(8.19))

# for string
print(float("1.23"))
```



```
C:\Users\praji\PyCharm
9.0
8.19
1.23
Process finished with...
```

What is Python Module

A [Python](#) module is a file containing Python definitions and statements. A module can define functions, classes, and variables. A module can also include runnable code.

Grouping related code into a module makes the code easier to understand and use. It also makes the code logically organized.

Create a Python Module

To create a Python module, write the desired code and save that in a file with **.py** extension. Let's understand it better with an example:

Example:

Let's create a simple `calc.py` in which we define two functions, one **add** and another **subtract**.

```
1 # A simple module, calc.py
2 def add(x, y):
3     return (x+y)
4
5 def subtract(x, y):
6     return (x-y)
```

Import module in Python

We can import the functions, and classes defined in a module to another module using the **import statement** in some other Python source file.

When the interpreter encounters an import statement, it imports the module if the module is present in the search path.

Note: A search path is a list of directories that the interpreter searches for importing a module.

For example, to import the module `calc.py`, we need to put the following command at the top of the script.

Syntax to Import Module in Python

```
import module
```

Importing modules in Python Example

Now, we are importing the `calc` that we created earlier to perform add operation.



```
# importing module calc.py
import calc

print(calc.add(10, 2))
```

Python Import From Module

Python's `from` statement lets you import specific attributes from a module without importing the module as a whole.

Import Specific Attributes from a Python module

Here, we are importing specific `sqrt` and `factorial` attributes from the `math` module.



```
1 # importing sqrt() and factorial from the
2 # module math
3 from math import sqrt, factorial
4
5 # if we simply do "import math", then
6 # math.sqrt(16) and math.factorial()
7 # are required.
8 print(sqrt(16))
9 print(factorial(6))
```

Import all Names

The * symbol used with the import statement is used to import all the names from a module to a current namespace.

Syntax:

```
from module_name import *
```

What does import * do in Python?

The use of * has its advantages and disadvantages. If you know exactly what you will be needing from the module, it is not recommended to use *, else do so.

```
1 # importing sqrt() and factorial from the
2 # module math
3 from math import *
4
5 # if we simply do "import math", then
6 # math.sqrt(16) and math.factorial()
7 # are required.
8 print(sqrt(16))
9 print(factorial(6))
```

Renaming the Python Module

We can rename the module while importing it using the keyword.

Syntax: *Import Module_name as Alias_name*

```
1 # importing sqrt() and factorial from the
2 # module math
3 import math as mt
4
5 # if we simply do "import math", then
6 # math.sqrt(16) and math.factorial()
7 # are required.
8 print(mt.sqrt(16))
9 print(mt.factorial(6))
```

```
# importing built-in module math
import math
```

```

# using square root(sqrt) function contained
# in math module
print("square root: ",math.sqrt(25))

# using pi function contained in math module
print("pi=",math.pi)

# Sine of 2 radians
print("sin=",math.sin(2))

# Cosine of 0.5 radians
print("cos=",math.cos(0.5))

# Tangent of 0.23 radians
print("tan=",math.tan(0.23))

# 1 * 2 * 3 * 4 = 24
print("factorial of 4=",math.factorial(4))

# importing built in module random
import random

# printing random integer between 0 and 5
print("random number from 0 and 5:",random.randint(0, 5))

# random number between 0 and 100
print("random number from 0 and 100:",random.random() * 100)

List = [1, 4, True, 800, "python", 27, "hello"]

# using choice function in random module for choosing
# a random element from a set such as a list
print("random value from list=",random.choice(List))

# importing built in module datetime
import datetime
from datetime import date
import time

# Returns the number of seconds since the
# Unix Epoch, January 1st 1970
print(time.time())

# Converts a number of seconds to a date object
print(date.fromtimestamp(454554))

```

```
C:\Users\praji\PycharmProjects\pythonProject11\.venv\Scr
square root: 5.0
pi= 3.141592653589793
sin= 0.9092974268256817
cos= 0.8775825618903728
tan= 0.23414336235146527
factorial of 4= 24
random number from 0 and 5: 0
random number from 0 and 100: 75.76204415060367
random value from list= hello
1736665026.0995529
1970-01-06
```

EXCEPTION HANDLING IN PYTHON

- Errors are the problems in a program due to which the program will stop the execution.

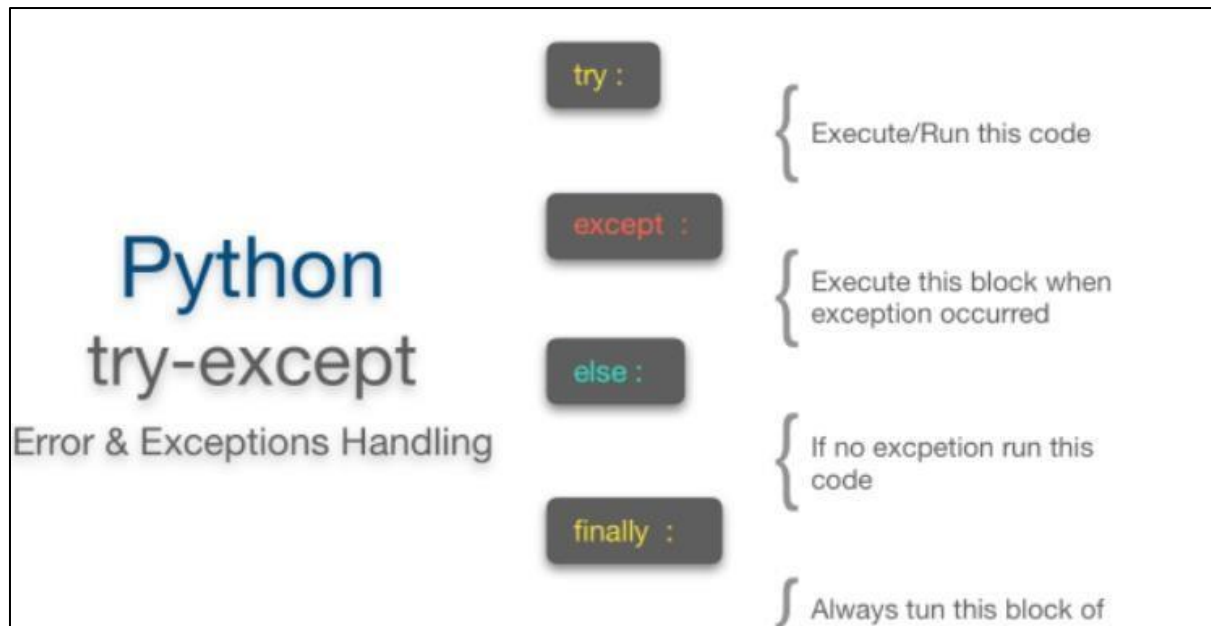
Two types of Error occurs in python.

- Syntax errors(parsing errors)
- Logical errors (Exceptions)
- An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

Syntax

Here is simple syntax of *try....except...else* blocks –

```
try:
    You do your operations here;
    .....
except ExceptionI:
    If there is ExceptionI, then execute this block.
except ExceptionII:
    If there is ExceptionII, then execute this block.
    .....
else:
    If there is no exception then execute this block.
```



```
try:
    f = open('somefile.txt', 'r')
    print(f.read())
    f.close()
except IOError:
    print('file not found')
```

```
try:
    num1, num2 = eval(input("Enter two numbers, separated by a comma : "))
    result = num1 / num2
    print("Result is", result)

except ZeroDivisionError:
    print("Division by zero is error !!")

except SyntaxError:
    print("Comma is missing. Enter numbers separated by comma like this 1, 2")

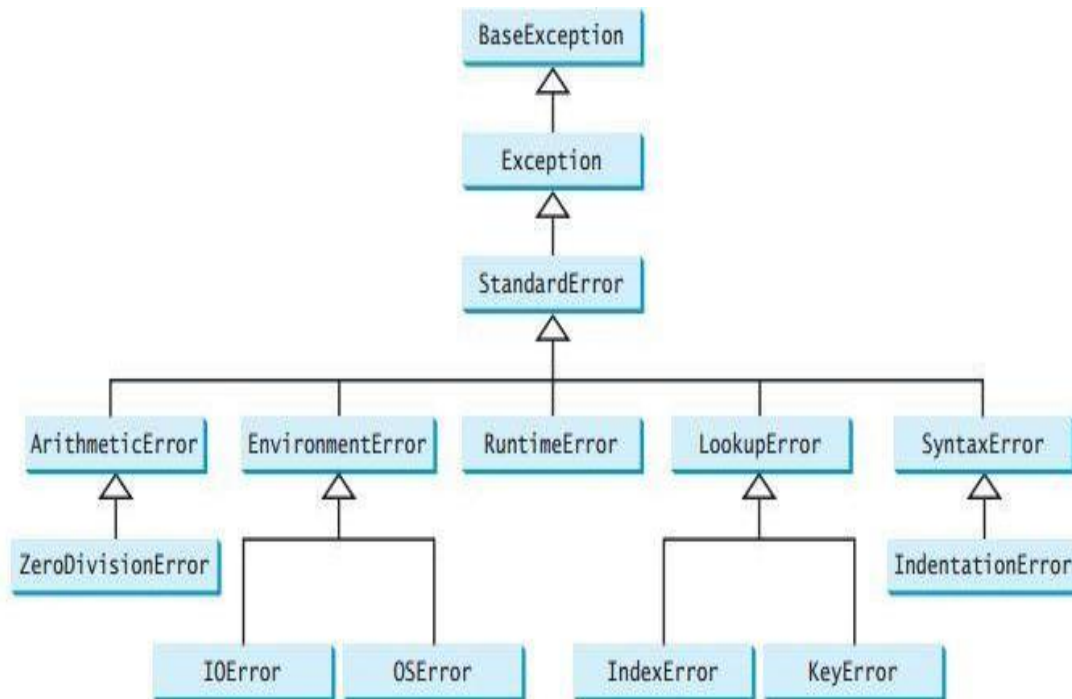
except:
    print("Wrong input")

else:
    print("No exceptions")

finally:
    print("This will execute no matter what")
```

COMMON EXCEPTIONS:

- **ZeroDivisionError:** Occurs when a number is divided by zero.
- **NameError:** It occurs when a name is not found. It may be local or global.
- **IndentationError:** If incorrect indentation is given.
- **IOError:** It occurs when Input Output operation fails.
- **EOFError:** It occurs when the end of the file is reached, and yet operations are being performed.



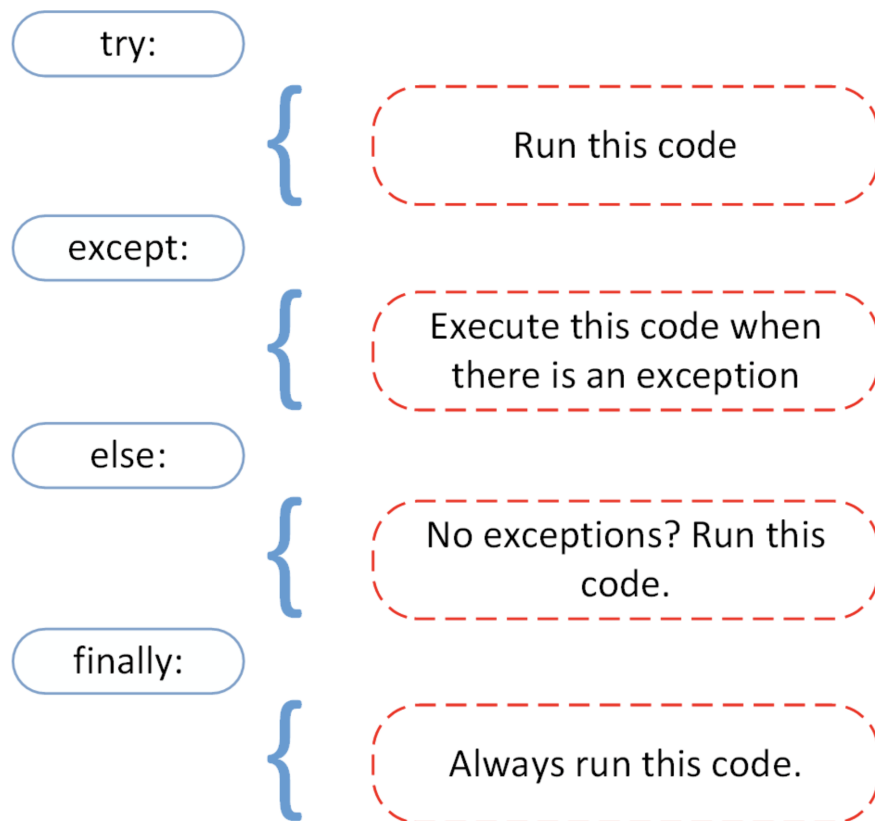
Class	Description
Exception	A base class for most error types
AttributeError	Raised by syntax <code>obj.foo</code> , if <code>obj</code> has no member named <code>foo</code>
EOFError	Raised if “end of file” reached for console or file input
IOError	Raised upon failure of I/O operation (e.g., opening file)
IndexError	Raised if index to sequence is out of bounds
KeyError	Raised if nonexistent key requested for set or dictionary
KeyboardInterrupt	Raised if user types ctrl-C while program is executing
NameError	Raised if nonexistent identifier used
StopIteration	Raised by <code>next(iterator)</code> if no element; see Section 1.8
TypeError	Raised when wrong type of parameter is sent to a function
ValueError	Raised when parameter has invalid value (e.g., <code>sqrt(-5)</code>)
ZeroDivisionError	Raised when any division operator used with 0 as divisor

EXAMPLE:

```
try:  
    a = int(input("Enter a:"))  
    b = int(input("Enter b:"))  
    c = a/b  
except:  
    print("Can't divide with zero")
```

Output:

```
Enter a:10  
Enter b:0  
Can't divide with zero
```



RAISE AN EXCEPTION:

- As a Python developer you can choose to throw an exception if a condition occurs.
- To throw (or raise) an exception, use the raise keyword.
- We can use raise to throw an exception if a condition occurs. The statement can be complemented with a custom exception.

- `x = -1`

`if x < 0:`

`raise Exception("Sorry, no numbers below zero")`

- `x = "hello"`

`if not type(x) is int:`

`raise TypeError("Only integers are allowed")`

`# Program to depict else clause with try-except`

`# Function which returns a/b`

```

def AbyB(a , b):
    try:
        c = ((a+b) / (a-b))
    except ZeroDivisionError:
        print "a/b result is 0"
    else:
        print c
# Driver program to test above function
AbyB(2.0, 3.0)
AbyB(3.0, 3.0)

```

OUTPUT:

```

-5.0

a/b result is 0

```

USER DEFINED EXCEPTION

- Programmers may name their own exceptions by creating a new exception class. Exceptions need to be derived from the Exception class, either directly or indirectly.

```

number = 10
# user guesses a number until he/she gets it right
while True:
    try:
        i_num = int(input("Enter a number: "))
        if i_num < number:
            raise ValueErrorTooSmallError
        elif i_num > number:
            raise ValueErrorTooLargeError
        break
    except ValueErrorTooSmallError:
        print("This value is too small, try again!")
        print()
    except ValueErrorTooLargeError:
        print("This value is too large, try again!")
        print()
print("Congratulations! You guessed it correctly.")

```

FILE IN PYTHON

- A file is a chunk of logically related data or information which can be used by computer programs.
- Files on most modern file systems are composed of three main parts:
- Header: metadata about the contents of the file (file name, size, type, and so on)
- Data: contents of the file as written by the creator or editor

- End of file (EOF): special character that indicates the end of the file

In Python, there is no need for importing external library to read and write files. Python provides an inbuilt function for creating, writing, and reading files.

- Python has several functions for creating, reading, updating, and deleting files.
- There are two types of files in Python

❖ Binary file

❖ Text file

Binary files in Python

Most of the files that we see in our computer system are called binary files.

Example:

1. **Document files:** .pdf, .doc, .xls etc.
2. **Image files:** .png, .jpg, .gif, .bmp etc.
3. **Video files:** .mp4, .3gp, .mkv, .avi etc.
4. **Audio files:** .mp3, .wav, .mka, .aac etc.
5. **Database files:** .mdb, .accde, .frm, .sqlite etc.
6. **Archive files:** .zip, .rar, .iso, .7z etc.
7. **Executable files:** .exe, .dll, .class etc.

Text files in Python

Text files don't have any specific encoding and it can be opened in normal text editor itself.

Example:

- **Web standards:** html, XML, CSS, JSON etc.
- **Source code:** c, app, js, py, java etc.
- **Documents:** txt, tex, RTF etc.
- **Tabular data:** csv, tsv etc.
- **Configuration:** ini, cfg, reg etc.

Python File Handling Operations

Most importantly there are 4 types of operations that can be handled by Python on files:

- Open
- Read
- Write
- Close

Other operations include:

- Rename
- Delete

FILE open() function

The Python file open function returns a file object that contains methods and attributes to perform various operations for opening files in Python.

```
file_object = open("filename", "mode")
```

Here,

- **filename:** gives name of the file that the file object has opened.
- **mode:** attribute of a file object tells you which mode a file was opened in.

File Modes:

MODES	DESCRIPTION
<r>	It opens a file in read-only mode while the file offset stays at the root.
<rb>	It opens a file in (binary + read-only) modes. And the offset remains at the root level.
<r+>	It opens the file in both (read + write) modes while the file offset is again at the root level.
<rb+>	It opens the file in (read + write + binary) modes. The file offset is again at the root level.
<w>	It allows write-level access to a file. If the file already exists, then it'll get overwritten. It'll create a new file if the same doesn't exist.
<wb>	Use it to open a file for writing in binary format. Same behavior as for write-only mode.
<w+>	It opens a file in both (read + write) modes. Same behavior as for write-only mode.
<wb+>	It opens a file in (read + write + binary) modes. Same behavior as for write-only mode.

<a>	It opens the file in append mode. The offset goes to the end of the file. If the file doesn't exist, then it gets created.
<ab>	It opens a file in (append + binary) modes. Same behavior as for append mode.
<a+>	It opens a file in (append + read) modes. Same behavior as for append mode.
<ab+>	It opens a file in (append + read + binary) modes. Same behavior as for append mode.

Syntax:

- **file_object = open(file_name, mode)**

EG

- **f = open("demofile.txt", "r")**

Closing A File In Python

In Python, it is not system critical to close all your files after using them, because the file will auto close after Python code finishes execution. You can close a file by using the `close()` method.

Syntax:

```
file_object.close();
```

Syntax:

```
file_object.close();
```

Example:

```
try:
    # Open a file
    fo = open("sample.txt", "wb")
    # perform file operations
finally:
    # Close opened file
    fo.close()
```

EXAMPLE:

Python program to demonstrate File Concept

```
file1 = open('IIBSC_CS.txt', 'w')
```

```
L = ["20UCS BATCH \n", "JAVA-PYTHON-DS \n", "VB-EVS \n"]
```

```
s = "WELCOME\n"
```

Writing a string to file

```
file1.write(s)
```

Writing multiple strings at a time

```
file1.writelines(L)
```

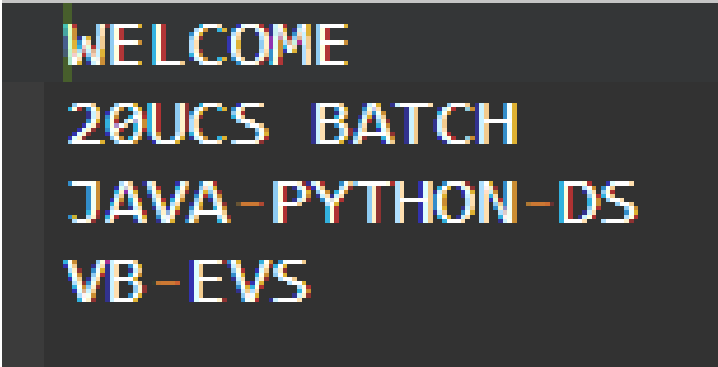
Closing file

```
file1.close()
```

```
file1 = open('IIBSC_CS.txt', 'r')
```

```
print(file1.read())
```

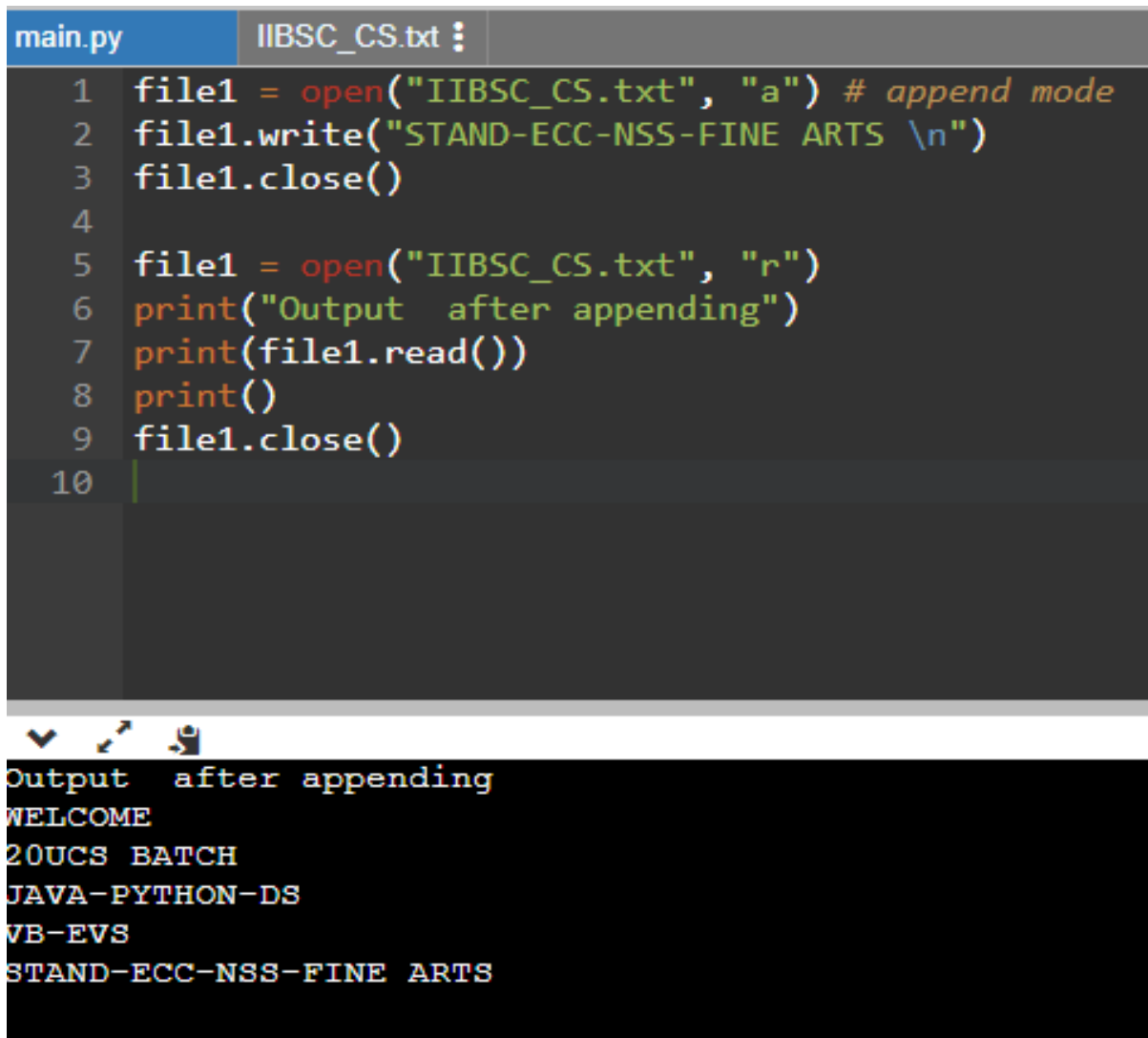
```
file1.close()
```



WELCOME
20UCS BATCH
JAVA-PYTHON-DS
VB-EVS

APPEND

```
# Python program to illustrate # Append  
# Append-adds at last  
file1 = open("IIBSC_CS.txt", "a") # append mode  
file1.write("STAND-ECC-NSS-FINE ARTS \n")  
file1.close()  
file1 = open("IIBSC_CS.txt", "r")  
print("Output after appending")  
print(file1.read())  
print()  
file1.close()
```



The screenshot shows a code editor with two tabs: 'main.py' and 'IIBSC_CS.txt'. The 'main.py' tab is active, displaying a Python script. The script opens 'IIBSC_CS.txt' in append mode, writes 'STAND-ECC-NSS-FINE ARTS' followed by a newline, and then closes the file. It then opens the same file in read mode, prints the output, and closes it. Below the code editor, a terminal window shows the output of the script, which is the content of the file after the append operation.

```
1 file1 = open("IIBSC_CS.txt", "a") # append mode
2 file1.write("STAND-ECC-NSS-FINE ARTS \n")
3 file1.close()
4
5 file1 = open("IIBSC_CS.txt", "r")
6 print("Output after appending")
7 print(file1.read())
8 print()
9 file1.close()
10
```

```
Output after appending
WELCOME
20UCS BATCH
JAVA-PYTHON-DS
VB-EVS
STAND-ECC-NSS-FINE ARTS
```

EXAMPLE:

Python program to count the number of lines in a text file

Opening a file

```
file = open("IIBSC_CS.txt","r")
```

```
Counter = 0
```

Reading from file

```
Content = file.read()
```

```
CoList = Content.split("\n")
```

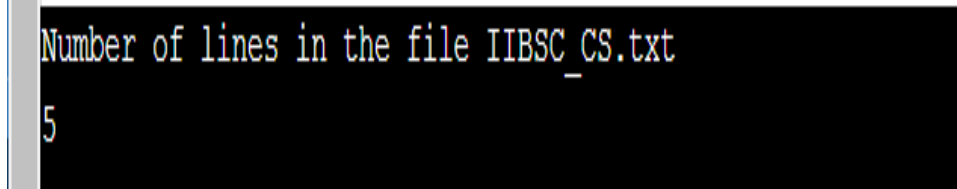
```
for i in CoList:
```

```
    if i:
```

```
        Counter += 1

print("Number of lines in the file IIBSC_CS.txt")

print(Counter)
```



```
Number of lines in the file IIBSC_CS.txt
5
```

EXAMPLE:

PROGRAM TO DISPLAY SUM OF DIGITS IN THE TEXT FILE

```
# Python program for writing to file

file = open('SUM.txt', 'w')

data = 'HELLO123 SUPER345'

file.write(data)

file.close()

# Python program for reading from file

h = open('SUM.txt', 'r')

content = h.readlines()

a = 0

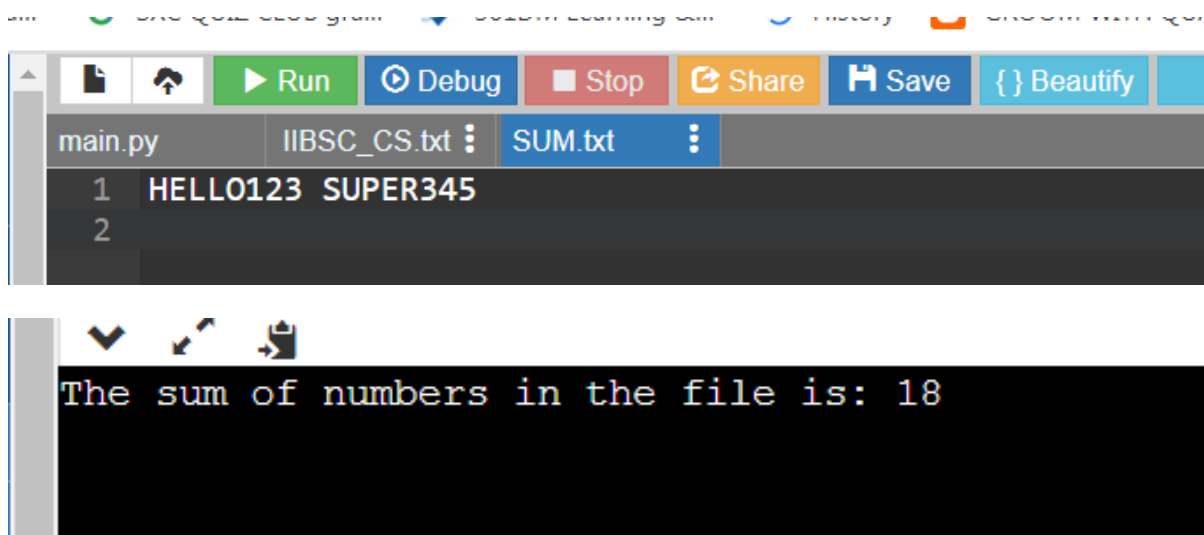
for line in content:

    for i in line:

        if i.isdigit() == True:

            a += int(i)

print("The sum of numbers in the file is:", a)
```

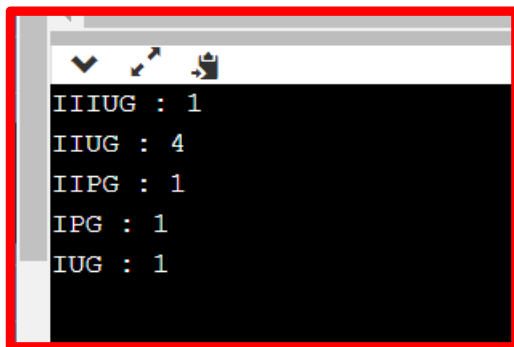
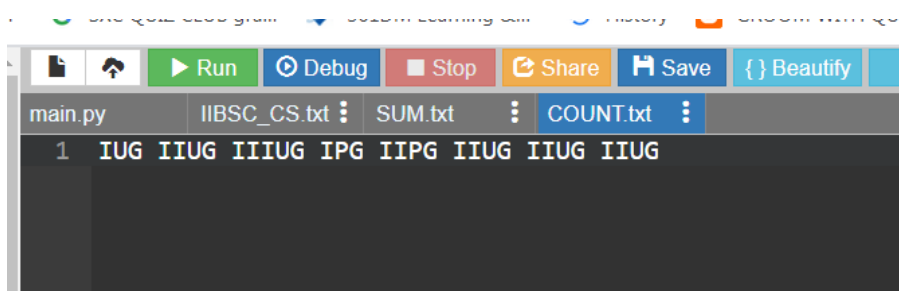
A screenshot of a code editor interface. The top toolbar includes buttons for Run (green), Debug (blue), Stop (red), Share (orange), Save (blue), and Beautify (light blue). Below the toolbar, a tab bar shows three files: main.py, IIBSC_CS.txt, and SUM.txt. The SUM.txt file is active, displaying two lines of code: 1 HELLO123 SUPER345 and 2. Below the code editor, a terminal window shows the output: 'The sum of numbers in the file is: 18'.

EXAMPLE

Count the number of occurrence of the words in text file

```
file = open('COUNT.txt', 'w')
data = 'IUG IIUG IIIUG IPG IIPG IIUG IIUG IIUG'
file.write(data)
file.close()
text = open("COUNT.txt", "r")
d = dict()
for line in text:
    line = line.strip()
    line = line.upper()
    words = line.split(" ")
    for word in words:
        if word in d:
            d[word] = d[word] + 1
        else:
            d[word] = 1
```

```
for key in list(d.keys()):  
    print(key, ":", d[key])
```



FILE METHODS:

<u>close()</u>	Closes the file
<u>detach()</u>	Returns the separated raw stream from the buffer
<u>fileno()</u>	Returns a number that represents the stream, from the operating system's perspective
<u>flush()</u>	Flushes the internal buffer
<u>isatty()</u>	Returns whether the file stream is interactive or not

read() Returns the file content

readable() Returns whether the file stream can be read or not

readline() Returns one line from the file

readlines() Returns a list of lines from the file

seek() Change the file position

seekable() Returns whether the file allows us to change the file position

tell() Returns the current file position

truncate() Resizes the file to a specified size

writable() Returns whether the file can be written to or not

write() Writes the specified string to the file

writelines() Writes a list of strings to the file

FILE:RANDOM ACCESS

1. The `tell()` method returns the current file position in a file stream.

file.tell()

```
f = open("demofile.txt", "r")
print(f.readline())
print(f.tell())
```

2. `seek()`

move the file pointer to another position. `fileObject.seek(offset,from_what)`

3. `offset` – A number of positions will move.

```
f = open("testFile.txt", "r")

f.seek(9)

print(f.readline())
```

Using the `mmap` module allows the user to randomly access locations in a file by mapping the file into memory. This is an alternative to using normal file operations.

```
import mmap

with open('filename.ext', 'r') as fd:
    # 0: map the whole file
    mm = mmap.mmap(fd.fileno(), 0)

    # print characters at indices 5 through 10
    print mm[5:10]
```

```
# print the line starting from mm's current position
print mm.readline()

# write a character to the 5th index
mm[5] = 'a'

# return mm's position to the beginning of the file
mm.seek(0)

# close the mmap object
mm.close()
```

Zippping and Unzipping files in Python

ZIP is an archive file format that supports lossless data compression. By lossless compression, it means that the

compression algorithm used allows the original data to be perfectly reconstructed from the compressed data.

Also, a **ZIP** file may contain one or more files or directories that may have been compressed.

Why do we need zip files?

- To reduce storage requirements.
- To improve transfer speed over standard connections.

```
# importing required modules
from zipfile import ZipFile
```

```
# specifying the zip file name
file_name = "my_python_files.zip"
```

```
# opening the zip file in READ mode
with ZipFile(file_name, 'r') as zip:
    # printing all the contents of the zip file
    zip.printdir()
```

```
# extracting all the files
print('Extracting all the files now...')
zip.extractall()
print('Done!')
```

Unzipping the File

```
import zipfile
def un_zipFiles(path):
    files=os.listdir(path)
    for file in files:
        if file.endswith('.zip'):
            filePath=path+'/'+file
            zip_file = zipfile.ZipFile(filePath)
            for names in zip_file.namelist():
                zip_file.extract(names,path)
            zip_file.close()
```